The `getResponseHeader` and `getAllResponseHeaders` methods can be used to read the HTTP headers on the response. `getResponseHeader` returns the value of a given response header, while `getAllResponseHeaders` returns all the response headers as a single string. There is one caveat to reading response headers on a cross-origin request. By default, CORS only allows the client code to read the following response headers:

- Cache-Control
- Content-Language
- Content-Type
- Expires
- Last-Modified
- Pragma

If the server sets any additional response headers that aren't in this list, the client won't be able to see them. But the server can also override this behavior by specifically indicating that these additional response headers should be visible to the client code. The details of how to do this are covered in chapter 5.

To understand when the `onload` handler fires, it's important to distinguish a successful response from a successful response status code. HTTP responses have an associated status code. A successful response usually has a status code of 200, although any status code in the 200 range signals a success. Status codes in the 300 range signal that the request is being redirected, while status codes of 400 or above signal an error (the 400 range is reserved for client errors while the 500 range is reserved for server errors).

Regardless of the underlying response status code, if the response makes it back to the browser, the `onload` event handler will fire. So although a request may fail due to a file not found (status code 404) or an internal server error (status code 500), the `onload` event handler will still fire. Figure 2.6 shows the relationship between the response and the response status code.

If the request fails for some other reason, the `onerror` event handler will fire. These are errors where the server doesn't send a valid response to the browser, or the server doesn't support CORS. The next section takes a look at how to use the `onerror` event handler.

HANDLING AN ERROR USING THE ONERROR HANDLER

The `onerror` event handler will fire if there is an issue with the request. This can happen if, for example, the servers powering the Flickr API are down and not responding. But the `onerror` event is particularly relevant to CORS because it fires if the server rejects the CORS request. If the Flickr API didn't support CORS, the `onerror` event handler would fire.

As we'll cover in the next part of this book, the server has many reasons to reject a CORS request. The server may allow cross-origin `GET` and `POST` requests, but not `PUT` or `DELETE` requests. In this case, if a client attempts a `PUT` request, the `onerror` handler will fire instead of the `onload` handler.